Iteración 1 – Base de Datos y Entorno

1. Objetivo

Configurar el entorno de desarrollo reproducible con Docker y crear la base de datos MySQL con las tablas base, dejando poblada información de prueba para las siguientes iteraciones.

2. Actividades

N.º	Actividad	Resultado
1	Crear el repositorio en GitHub, ramas `main` y `develop`, `.gitignore` y `.env.example`.	Repositorio inicial con reglas de ramas.
2	Crear el proyecto Laravel (`backend/`) y el proyecto React (`frontend/`).	Proyectos que compilan de forma independiente.
3	Crear `docker-compose.yml` con MySQL, phpMyAdmin, backend y frontend.	Servicios levantados con `docker compose up`.
4	Crear las migraciones de Laravel para las tablas base (`usuarios`, `vehiculos`, `incidencias`, `rutas`, `disposicion_final`, `contratos`).	Esquema creado con `php artisan migrate`.
5	Crear los seeders: usuarios por rol, vehículos, incidencias de ejemplo.	Datos de prueba con `php artisan db:seed`.
6	Configurar ESLint, Prettier y Laravel Pint.	Estándares de código activos.

3. Configuración de Docker Compose

Ejemplo de `docker-compose.yml` (los valores sensibles se leen desde `.env`, nunca se escriben en el archivo):

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: ${DB_DATABASE}
      MYSQL_USER: ${DB_USERNAME}
      MYSQL_PASSWORD: ${DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    volumes: [db_data:/var/lib/mysql]
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      retries: 10
  phpmyadmin:
    image: phpmyadmin:5
    environment: { PMA_HOST: db }
    ports: ["8080:80"]
    depends_on: [db]
  backend:
    build: ./backend
    env_file: ./backend/.env        # DB_HOST=db
    ports: ["8000:8000"]
    depends_on:
      db: { condition: service_healthy }
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
```

```
    depends_on: [backend]
volumes:
  db_data:
```

Orden de migraciones

- `usuarios` → `vehiculos` (sin dependencias).
- `incidencias` (FK a usuarios) → `rutas` (FK a usuarios y vehículos) → `disposicion_final` (FK a vehículos y rutas) → `contratos` (FK a usuarios).

Comandos de verificación

```
docker compose up -d --build
docker compose exec backend php artisan migrate --seed
docker compose exec db mysql -u$DB_USERNAME -p -e "SHOW TABLES" $DB_DATABASE
```

4. Verificación

ID	Caso de prueba	Entrada / condición	Resultado esperado	Resultado
IT1-01	Levantar el entorno	`docker compose up -d`	Los 4 servicios inician sin errores.	Pendiente
IT1-02	Migraciones	`php artisan migrate`	Se crean las 6 tablas con sus claves foráneas.	Pendiente
IT1-03	Seeders	`php artisan db:seed`	Existen usuarios de los 4 roles, vehículos e incidencias.	Pendiente
IT1-04	Conexión Laravel–MySQL	`DB_HOST=db`	La API responde y consulta la BD.	Pendiente
IT1-05	Repositorio limpio	Revisar `git ls-files`	No hay archivos `.env` versionados.	Pendiente

5. Problemas y soluciones

Problema / riesgo	Solución / mitigación	Estado
Error de conexión entre Laravel y Docker.	Se configuró el host como `db` (nombre del servicio) en lugar de `localhost`.	Resuelto (ejemplo)
Laravel arranca antes de que MySQL esté listo y falla la migración.	Se agregó `healthcheck` a MySQL y `depends_on` con `condition: service_healthy`.	Mitigación
Puertos 3306 u 8080 ocupados en la máquina local.	No exponer 3306 al host o cambiar el puerto publicado.	Mitigación
Permisos de `storage/` y `bootstrap/cache`.	Ajustar propietario/permisos en el Dockerfile del backend.	Mitigación

6. Criterios de cierre

- Entorno levantado con un solo comando.
- Esquema y datos de prueba creados.
- Documentación de la iteración con capturas.